INFORMATICS INSTITUTE OF TECHNOLOGY | UNIVERSITY OF WESTMINSTER

CW1: INDIVIDUAL SUBMISSION

3D Interactive Media Development

(5MMCS007C)

Name	Sathsindu Induwara
Westminster ID	w2181825
IIT ID	20241422
Submission	CW1 - INDIVIDUAL SUBMISSION
Tutorial Group	G8

01. Individual contribution

Rabbit Survival is a Unity prototype in which a white rabbit follows a forest trail, avoids patrolling hunters and crosses a bridge to reach a second rabbit inside a cave. The Student 4 allocation was the Tutorial 04 key trigger and exploding brick array. In the group scene, the array forms the entrance to the final cave. The Student 4 implementation also includes a rabbit walk button, two hunter patrols and a reunion trigger. Together, these features give the route a visible obstacle, a way to open it and a clear end condition.

Key input and brick animation

The wall is the 8BrickArrayExplode object in RabbitSurvivalLEGO. In Student4ExplodeKey.cs, Awake gets the Animator and creates an Open Cave input action for the number 1 key on the keyboard. The action calls TryOpen when performed. The method checks three conditions before playing the clip. The wall must be closed, it must be idle, and the rabbit must be within the configured three Unity units. The distance check uses squared distance and compares it with the square of the activation range. This lets the player use the key when they reach the entrance, where the scene displays the instruction.

Student4ExplodeKey.cs • Key binding

```
private void Awake()
{
    brickAnimator = GetComponent<Animator>();
    openCave = new InputAction("Open Cave", InputActionType.Button, "<Keyboard>/1");
    openCave.performed += TryOpen;
}
```

Student4ExplodeKey.cs • Opening checks and animation

```
private void TryOpen(InputAction.CallbackContext context)
{
    if (IsOpen || opening ||
        (rabbit.position - transform.position).sqrMagnitude > activationDistance * activationDistance)
        return;

    opening = true;
    brickAnimator.Play("Base Layer.Explode", 0, 0f);
}
```

Once the checks pass, the script sets opening to true and plays Explode from the beginning. The Animator starts in Rest and transitions from Explode to Explode Rest when the clip completes. Update checks for that final state before disabling the BoxCollider that blocks the entrance. It then sets IsOpen to true and clears opening. The two flags cover different stages. During the animation, opening prevents repeat presses from restarting it. After the animation, IsOpen preserves the completed state. Reading the Animator state also ties access to the end of the clip if its duration changes.

Student4ExplodeKey.cs • Animation completion and doorway collider

```
private void Update()
{
    if (!opening || !brickAnimator.GetCurrentAnimatorStateInfo(0).IsName("Base Layer.Explode Rest"))
        return;

    doorway.enabled = false;
    IsOpen = true;
    opening = false;
}
```

The component enables its input action in OnEnable, disables it in OnDisable and disposes of it in OnDestroy. That keeps the keyboard action active while the wall component is enabled and cleans it up when the object is removed. The rabbit reference, doorway reference and activation distance are exposed on the component. These settings let the group connect the script to the player and the entrance directly in the scene.

Rabbit walk button

The Play rabbit walk button uses Unity's On Click event to call PlayWalk in Student4RabbitWalk.cs. The method gets the Animator on the rabbit object and starts the Walk state from time zero. The clip changes the vertical position and scale of the visual child, producing a short bounce and squash. The character controller and camera stay on the rabbit root. This keeps the animation preview separate from the position used for moving, colliding with the environment and checking nearby interactions. The button can play the preview again on another click.

Student4RabbitWalk.cs • Button event handler

```
public void PlayWalk()
{
    GetComponent<Animator>().Play("Base Layer.Walk", 0, 0f);
}
```

The recorded scene uses the white rabbit for this animation. The brief also specifies an animated LEGO minifigure with a UI button, so evidence of any separate minifigure work would need its own clip or screenshot. The walk test recorded in the handover applies to the white rabbit setup shown in this scene.

02. Group integration and playtesting

Hunter patrols

The Olive Ranger and Orange Scout each have four waypoints assigned in the scene. Student4HunterPatrol.cs reads the target position and uses Vector3.MoveTowards to step toward it each frame. Multiplying speed by Time.deltaTime makes the step depend on elapsed time. Once the hunter gets close enough, the index advances. Taking the remainder after division by the waypoint count returns the index to zero after the last point. Both hunters can use the same script with their own routes and speeds.

Student4HunterPatrol.cs • Waypoint movement and loop

```
transform.position = Vector3.MoveTowards(transform.position, target, speed * Time.deltaTime);
if ((transform.position - target).sqrMagnitude < 0.0025f)
    nextWaypoint = (nextWaypoint + 1) % waypoints.Length;
```

The patrol reads IsHidden from Student 3's rabbit controller, so the bushes on the route affect the catch check. The check also skips a rabbit outside catch range and stops after a win. Within range, a linecast from the hunter toward the rabbit checks for an obstacle. A successful catch sends the rabbit back to the start and shows a message suggesting the bushes for cover. This interaction connects the patrol to the group's movement and hiding system through the existing player controller.

The linecast ignores trigger volumes, allowing the script to check for a solid object between the hunter and the player. During a catch, the script temporarily disables the rabbit's CharacterController, moves and rotates the rabbit to the starting transform, and re-enables the controller. This keeps the normal movement

component available when play resumes. The patrol then calls ShowCaught on the goal component to use the same status text as the cave instructions.

Student4HunterPatrol.cs • Hiding, distance and line-of-sight checks

```
Vector3 eyes = transform.position + Vector3.up * 0.85f;
Vector3 rabbitCenter = rabbitBody.bounds.center;
if (rabbit.IsHidden || caveGoal.Won || (rabbitCenter - eyes).sqrMagnitude > catchDistance * catchDistance)
    return;

if (Physics.Linecast(eyes, rabbitCenter, out RaycastHit hit, ~0, QueryTriggerInteraction.Ignore)
    && hit.collider != rabbitBody)
    return;
```

Cave goal trigger

At the cave, Student4CaveGoal.cs updates the player's instructions based on distance and wall state. Near the closed entrance it shows "Press 1 to open the cave." Once the wall opens, the instruction directs the player to the second rabbit. OnTriggerEnter and OnTriggerStay call TryWin when a collider enters or remains inside the goal area. The method checks that it is the white rabbit, that the wall reports IsOpen and that Won is still false. A successful check sets Won and displays "You found the second rabbit!"

Student4CaveGoal.cs • Trigger events and win check

```
private void OnTriggerEnter(Collider other) => TryWin(other);

private void OnTriggerStay(Collider other) => TryWin(other);

private void TryWin(Collider other)
{
    if (Won || other != rabbit || !caveWall.IsOpen)
        return;

    Won = true;
    status.text = "You found the second rabbit!";
}
```

Recorded Play Mode checks

The Member 4 handover describes checks of individual features in Play Mode. The button was invoked and the white rabbit's visual transform was sampled. The recorded vertical change was 0.10 Unity units. Both hunters changed position between samples. These observations show that the button moved the visual model and that both patrol scripts were active. They provide a starting point for checking a full waypoint loop or how the hunters behave around hiding spots.

For the goal check, the rabbit was placed at the reunion point while the wall was closed. The win stayed inactive. The test then sent the 1 key near the entrance, confirmed that the doorway collider disabled and moved the rabbit into the goal. The win message appeared. Testing the goal before and after opening the gate checked that the shared IsOpen value controlled completion as intended. Figure 1 shows the starting trail. Figure 2 shows the visible result of reaching the goal.

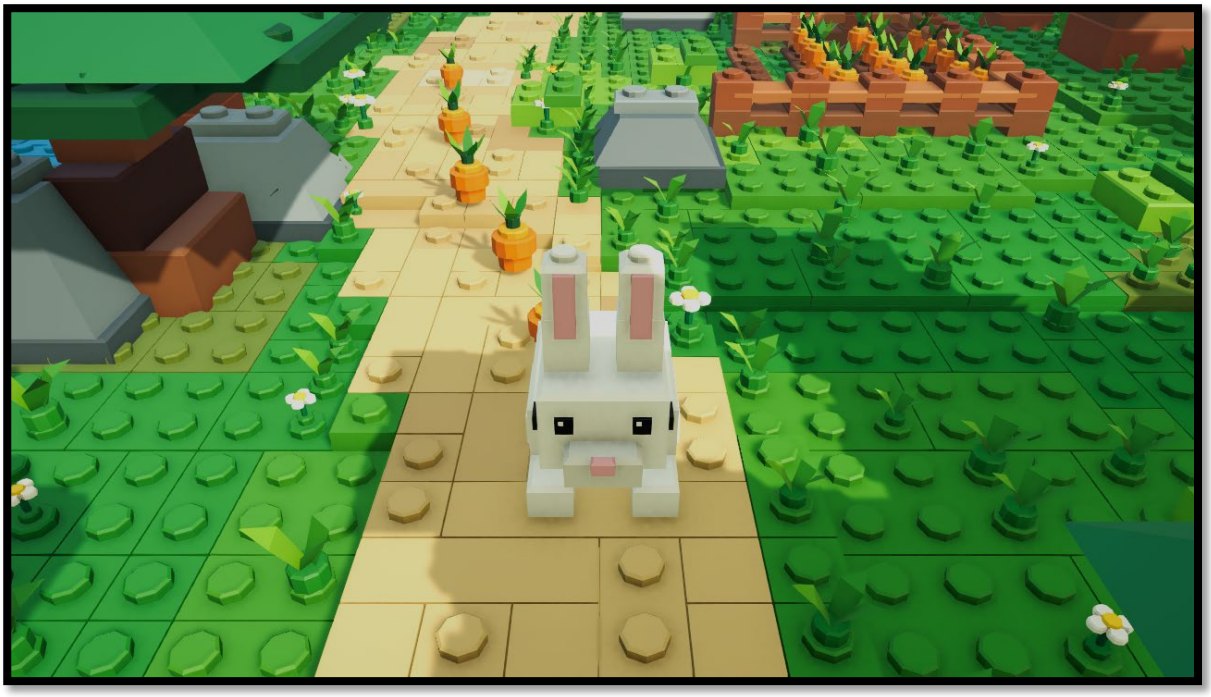


Figure 1. Starting trail.



Figure 2. Cave win message.

03. Evaluation and AI use

Evaluation and further testing

The strongest design choice in the cave sequence is tying the physical barrier to an observable animation state. If the collider were removed at the key press, the rabbit could move into the entrance while the bricks still looked solid. Waiting for Explode Rest lets the visual and collision states change in the expected order. The separate goal trigger then makes opening the wall a step in the route. The player still has to reach the second rabbit before the game marks the objective complete.

The recorded checks establish that the main steps work under the tested conditions. A useful follow-up is a continuous run from the trail through the bushes, across the bridge and into the cave. At the entrance, I would press 1 just outside the prompt range, repeat the press inside the range and press again during the animation. I would check that the wall stays closed at the first distance, opens at the second and continues playing after the repeated press. A capture showing the prompt, the Animator state and the collider would make the transition easier to verify from the evidence.

The patrol approach suits a short level because the route is predictable and easy to adjust in the Inspector. A hunter moves directly between assigned waypoints, so each segment needs to be checked against nearby terrain when the group edits the map. The current catch logic combines a distance check, the rabbit's hiding state and a linecast. A playtest could compare an exposed approach with an approach behind cover and use the result to judge whether players understand when they can be caught. This would test the behaviour a player experiences beyond the recorded movement samples.

For each follow-up run, the evidence should record the scene version, input, expected result and observed result. Keeping those four details with a screenshot would help a teammate reproduce the test after changing the trail, waypoint positions or cave entrance.

AI assistance and evidence

I used Codex to help write parts of the code for the rabbit walk button and hunter patrols. It also helped me set up the cave wall's animation states and check the cave sequence in Unity. I used it to get a script for the video.